



Tribhuvan University

Faculty of Humanities and Social Sciences

Kirtipur, Kathmandu

A Technical Writing

On

Introduction to Web Development and Career Roadmap for 2026

Submitted by

Name: Bishal Rijal

Roll No: 6 (Six)

Submitted to

Department of Computer Application

*In partial fulfilment of the requirements for the Masters in Computer
Application*

Patan Multiple Campus

Patandhoka, Lalitpur

Department of MCA

Affiliated to Tribhuvan University

ABSTRACT

Web development is a critical area of information technology encompassing the design, development, deployment, and maintenance of websites and web applications. As digital services continue to proliferate across education, business, communication, finance, healthcare, and entertainment sectors, students increasingly require a structured understanding of web technologies and the skills necessary to enter the software development field.

This technical writing, entitled "Introduction to Web Development and Career Roadmap for 2026," has been developed to provide beginners and students with structured technical information about web development and a practical learning pathway. The report introduces fundamental concepts including client-server architecture, frontend development, backend development, databases, APIs, development tools, testing, deployment, and web security.

The also presents a structured career roadmap beginning with web fundamentals and progressing through frontend, backend, and full-stack development. Particular attention is given to practical project development, version control, portfolio preparation, communication skills, problem solving, internship preparation, and continuous learning.

The technical report is designed as the primary information source for an educational webinar. The webinar uses presentation slides, demonstrations, examples, and an interactive question-and-answer session to communicate the technical content effectively. Audience analysis is incorporated to ensure that the technical information is understandable to +2 students, bachelor-level students, beginners, and individuals interested in software development.

The overall purpose of this writing is not only to explain web development but also to demonstrate the principles of technical communication by presenting complex technical information in a structured, clear, concise, and audience-oriented form.

Keywords: Web Development, Frontend, Backend, Database, Full-Stack Development, Career Roadmap, Technical Writing, Webinar, Software Development

Table of Contents

ABSTRACT	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS.....	vii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Purpose of the Technical Writing.....	1
1.3 Problem Statement.....	2
1.4 Objective	2
1.4.2 Specific Objectives.....	2
1.5 Target Audience	3
1.6 Scope of the Webinar	4
1.7 Significance of the Project.....	4
1.8 Limitations.....	5
CHAPTER 2: FUNDAMENTALS OF WEB DEVELOPMENT	6
2.1 What is Web Development.....	6
2.2 Website vs. Web Application	6
2.3 How the Web Works	6
2.3.6 APIs Introduction	12
CHAPTER 3: WEB DEVELOPMENT TECHNOLOGIES	15
3.1 Frontend Technologies	15
3.2 Backend Technologies.....	15
3.3 Databases and Version Control	15
3.4 Development Tools and Deployment	16
CHAPTER 4: CAREER ROADMAP FOR WEB DEVELOPMENT IN 2026.....	17
4.1 Roadmap Overview	17
4.2 Stages 1–5: Foundations to Backend.....	18
4.3 Stages 6–10: Database to Continuous Learning.....	18
CHAPTER 5: SKILLS AND CAREER OPPORTUNITIES	20
5.1 Technical and Professional Skills.....	20
5.2 Career Opportunities	20
5.3 Career Preparation	20
CHAPTER 6: METHODOLOGY AND IMPLEMENTATION	21

6.1 Research Method and Information Gathering	21
6.2 Webinar Implementation	21
6.4 Resources and Risk Management.....	22
CHAPTER 7: CONCLUSION AND RECOMMENDATIONS	23
7.1 Conclusion.....	23
7.2 Recommendations	23
Chapter 8: REFERENCES.....	25

LIST OF FIGURES

<i>Figure 1: Client-Server Architecture</i>	<i>7</i>
<i>Figure 2 : Components of Web Development.....</i>	<i>9</i>
<i>Figure 3: HTTP vs HTTPS.....</i>	<i>10</i>
<i>Figure 4: DNS Resolution Flow</i>	<i>11</i>
<i>Figure 5: Three-Tier Architecture.....</i>	<i>12</i>
<i>Figure 6: API as a Bridge Between Frontend and Backend</i>	<i>14</i>
<i>Figure 7: Web Development Career Roadmap</i>	<i>17</i>
<i>Figure 8: Full-Stack System</i>	<i>19</i>

LIST OF TABLES

Table 1: Target Audience and Information Needs	3
Table 2: Components of Web Development and Their Roles.....	8
Table 3: Types of APIs and Their Characteristics.....	12
Table 4: REST API Methods and Endpoints.....	13
Table 5: Tentative Webinar Schedule	21
Table 6: Webinar Risk Management Plan.....	22

LIST OF ABBREVIATIONS

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CSS	Cascading Style Sheets
DNS	Domain Name System
DOM	Document Object Model
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
OWASP	Open Worldwide Application Security Project
QA	Quality Assurance
SQL	Structured Query Language
SSL	Secure Sockets Layer
TLS	Transport Layer Security
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

CHAPTER 1: INTRODUCTION

1.1 Background

The internet has become an essential part of modern society, supporting activities such as education, business, communication, banking, healthcare, entertainment, and government services. Websites and web applications allow users to access information and digital services through various internet-enabled devices.

Web development is the process of creating and maintaining websites and web applications. It involves several areas, including frontend development, backend development, databases, APIs, testing, deployment, security, and maintenance.

For beginners, web development can be challenging because of the wide range of programming languages, frameworks, libraries, and tools available. This writing provides a structured introduction to web development and a practical career roadmap for 2026, helping learners understand how different technologies work together to build complete applications.

The writing also highlights the importance of technical communication, documentation, presentation, and teamwork, as developers need both technical and communication skills for professional software development.

1.2 Purpose of the Technical Writing

The primary purpose of this technical writing is to collect, organize, simplify, and communicate information about web development for an audience with limited or intermediate technical knowledge.

This has two interconnected outputs:

1. A technical report containing structured information about web development.
2. An educational webinar based on the technical report.

The report serves as a reference document, while the webinar provides an interactive method of communicating the information.

1.3 Problem Statement

The rapid growth of web technologies has created both opportunities and challenges for beginners. Students interested in web development can access thousands of tutorials, courses, videos, documentation pages, and online communities. However, the sheer volume of information can make it difficult to determine what should be learned first and how different technologies relate to one another.

Beginners may also become confused between frontend, backend, databases, frameworks, libraries, APIs, hosting, and deployment. Learning technologies without understanding their relationships can result in an unstructured learning process.

Another challenge is that web development technologies evolve over time. Therefore, learners need a strong foundation in fundamental technologies and a learning strategy that allows them to adapt to changing tools and frameworks.

This addresses these issues by organizing web-development concepts into a structured learning and career roadmap.

1.4 Objective

1.4.1 General Objective

The general objectives are:

To provide participants with a fundamental understanding of web development and guide them through a structured career roadmap for becoming a web developer in 2026.

1.4.2 Specific Objectives

The specific objectives are:

- To introduce the fundamental concepts of web development.
- To explain how websites and web applications function.
- To explain client-server architecture.
- To distinguish between frontend, backend, and database systems.
- To introduce HTML, CSS, and JavaScript.
- To introduce modern frontend and backend technologies.

- To explain the role of APIs in web applications.
- To introduce version control and development tools.
- To explain the general web development workflow.
- To develop a structured career roadmap for aspiring web developers.
- To identify important technical and professional skills.
- To discuss possible career opportunities.
- To develop technical communication skills through a webinar.
- To demonstrate technical information using presentation slides and practical examples.

1.5 Target Audience

The primary target audience consists of students and beginners interested in information technology and web development.

Table 1: Target Audience and Information Needs

Audience	Technical Level	Information Need
+2 students	Beginner	Basic introduction and career options
Bachelor-level students	Beginner/Intermediate	Technologies and career roadmap
IT beginners	Beginner	Structured learning path
Aspiring developers	Beginner/Intermediate	Skills, projects, and career preparation
Career-oriented learners	Intermediate	Portfolio, internship, and job preparation

The technical content is therefore designed using simple explanations before introducing more technical terminology.

1.6 Scope of the Webinar

The webinar covers the fundamental concepts of web development, including:

- Web fundamentals
- Client-server architecture
- Frontend development
- Backend development
- Database systems
- APIs
- HTML
- CSS
- JavaScript
- Frontend frameworks and libraries
- Backend technologies
- Git and version control
- Development tools
- Testing and debugging
- Deployment
- Web security
- Career development
- Portfolio development
- Webinar communication

The project does not attempt to provide complete training in every programming language or framework. Instead, it provides a structured foundation and learning direction.

1.7 Significance of the Project

This project is significant because it combines technical knowledge with technical communication.

For students, the report provides a structured way to understand web development. For beginners, it reduces the complexity of choosing technologies by presenting them in a logical sequence.

For the Technical Writing course, the project demonstrates:

- Audience analysis
- Technical research
- Information organization
- Formal technical writing
- Presentation preparation
- Webinar planning
- Technical demonstration
- Documentation
- Communication skills

The project can also serve as a reference for students who want to begin learning web development.

1.8 Limitations

The project has the following limitations:

- Web technologies change rapidly, so specific tools may change after the report is prepared.
- The report provides an introduction rather than complete professional training.
- Career opportunities may differ according to country, organization, experience, and specialization.
- The proposed roadmap is a general learning framework rather than a fixed sequence that every learner must follow.
- The webinar duration limits the amount of practical coding that can be demonstrated

CHAPTER 2: FUNDAMENTALS OF WEB DEVELOPMENT

2.1 What is Web Development

Web development refers to the process of creating websites and web applications that operate through web browsers and internet technologies.

A basic web page can be created using HTML and CSS, while JavaScript can be used to add interaction and dynamic behaviour. More complex applications commonly require server-side programming, databases, APIs, authentication, testing, and deployment.

Web development can broadly be divided into:

- Frontend development
- Backend development
- Database development
- Full-stack development

2.2 Website vs. Web Application

A website is primarily informational and static (or semi-static). Its main purpose is to display content to visitors. e.g., a company profile, a blog, a news page. Users mostly *read*.

A web application is interactive and dynamic. It allows users to *do* things; log in, post content, buy products, chat, manage data. It behaves more like software running in a browser. The line between the two has blurred (many modern sites are both), but the core distinction is interactivity and functionality vs. information delivery.

2.3 How the Web Works

At its core, the web works through a request–response cycle between a client (browser) and a server (a computer hosting the website). When you type a URL or click a link:

1. The browser resolves the domain name to an IP address via DNS.
2. It sends an HTTP/HTTPS request to that IP.
3. The server processes the request (possibly querying a database).
4. The server sends back an HTTP response (HTML, CSS, JS, JSON, images).

5. The browser renders the response into the page you see.

This entire interaction is built on the Client-Server Architecture

2.3.1 Client-Server Architecture

Client-Server Architecture is a model where tasks are divided between two parties:

- **Client** — the device/software that requests services or resources (browser, mobile app).
- **Server** — the device/software that provides those services or resources (web server, application server, database server)

The client sends a request; the server processes it and returns a response. This separation allows centralized data management, security, scalability, and multi-user access.

Common server types include:

- **Web server** — serves HTML/CSS/JS (Apache, Nginx)
- **Application server** — runs business logic (Node, Django)
- **Database server** — stores and retrieves data (MySQL, MongoDB)

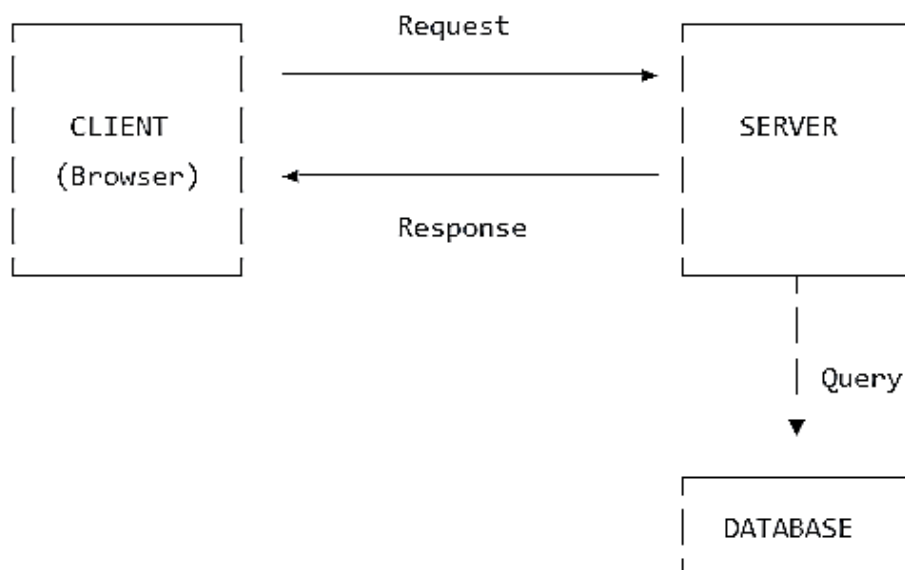


Figure 1: Client-Server Architecture

2.3.2 Components of Web Development

Web development is composed of multiple interconnected components that work together to deliver a functional web experience. These components span the client side, server side, and data layer, along with the protocols and infrastructure that connect them.

Table 2: Components of Web Development and Their Roles

Component	Role	Examples
Frontend	User interface & interaction	HTML, CSS, JS, React
Backend	Server logic & APIs	Node.js, Python, PHP
Database	Data storage & retrieval	MySQL, MongoDB
Web Server	Serves content	Apache, Nginx
Protocol	Communication rules	HTTP, HTTPS
DNS	Name resolution	Domain → IP
API	Interface between layers	REST, GraphQL

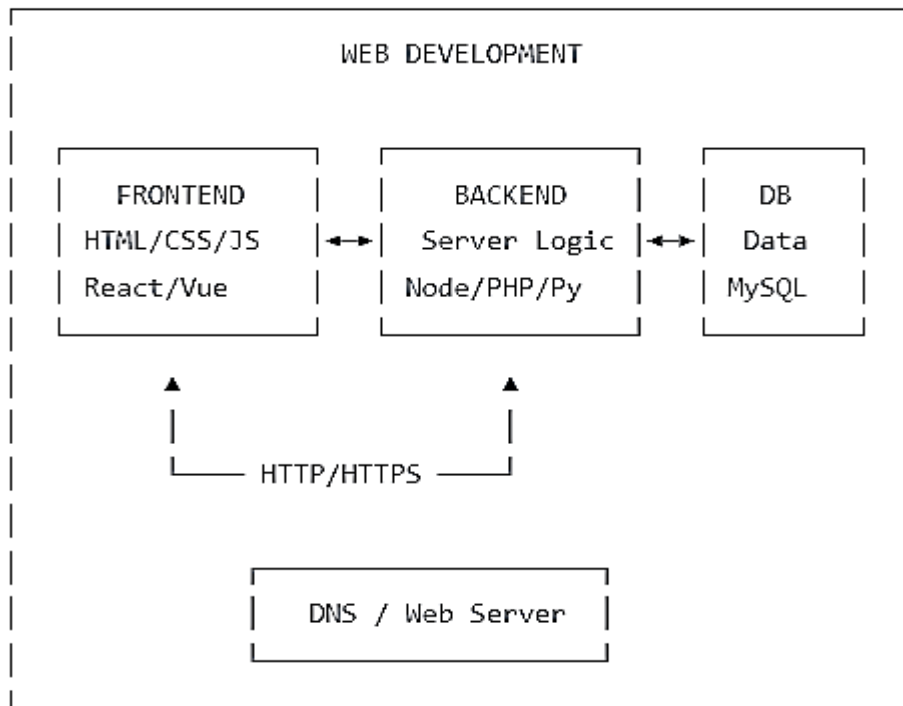


Figure 2 : Components of Web Development

2.3.3 HTTP and HTTPS

HTTP (HyperText Transfer Protocol) is the protocol used to transfer data between client and server on the web. It defines how requests are formatted, how responses are returned, and how errors are communicated.

HTTPS (HTTP Secure) is HTTP encrypted with SSL/TLS. It protects data from eavesdropping, tampering, and man-in-the-middle attacks. Modern browsers mark HTTP sites as "Not Secure." HTTPS is now the standard for all websites.

Key characteristics of HTTP:

- Stateless — each request is independent
- Request/Response based — client asks, server answers
- Uses methods — GET POST, PUT, DELETE, etc.
- Uses status codes — 200 OK, 404 Not Found, 500 Server Error
- Runs on port 80 (HTTP) or port 443 (HTTPS)

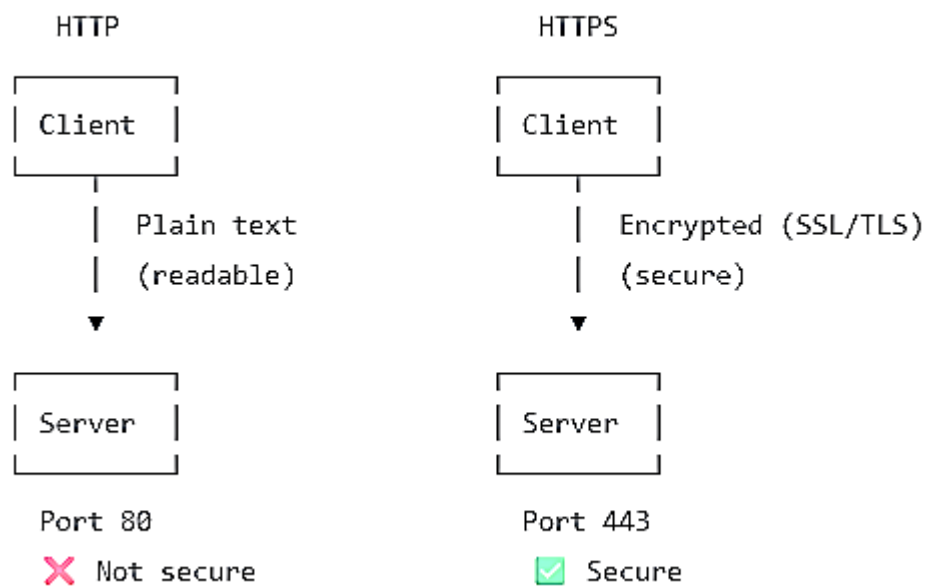


Figure 3: HTTP vs HTTPS

2.3.4 Domain Name and DNS

A domain name is a human-readable identifier for a web resource (for example, Network google.com). devices, however, are addressed by IP addresses (for example, DNS 142.250.190.46). (Domain Name System) is the distributed naming system that translates domain names into IP addresses. It is commonly described as the "phonebook of the internet."

DNS Resolution Process

1. The user enters a domain name in the browser.
2. The browser checks its local cache; if the record is absent, it queries a DNS resolver, typically provided by the ISP.
3. The resolver queries, in sequence, the root servers, the TLD servers (for example, .com), and the authoritative name servers for the domain.
4. The authoritative name server returns the corresponding IP address.
5. The browser establishes an HTTP/HTTPS connection to that IP address.

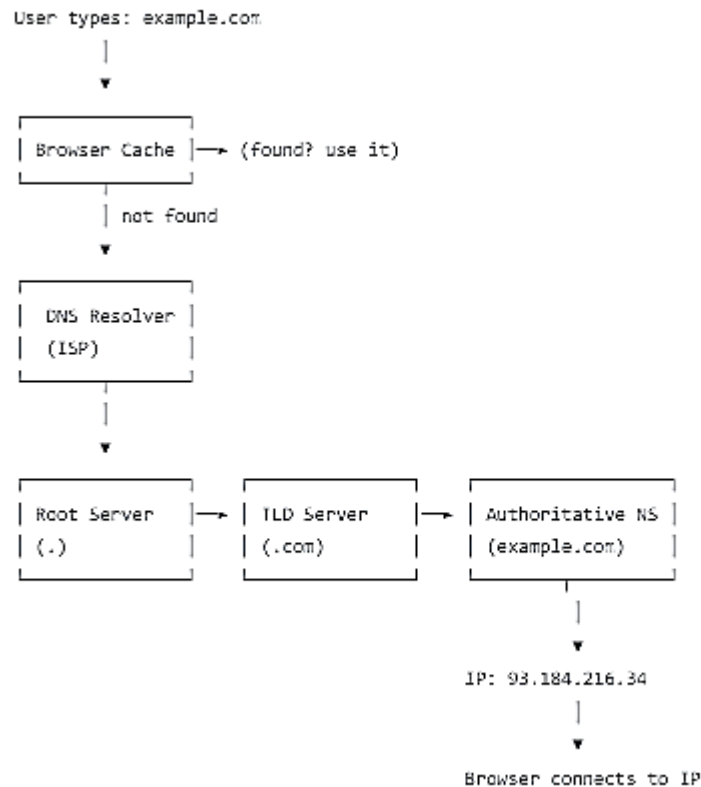


Figure 4: DNS Resolution Flow

2.3.5 Frontend, Backend, and Database Overview

Web development is typically divided into three layers:

- **Frontend** — the client-side, what users see and interact with. Built with HTML, CSS, JavaScript, and frameworks like React, Vue, Angular.
- **Backend** — the server-side, handling logic, authentication, requests, and business rules. Built with Node.js, Python (Django/Flask), PHP (Laravel), Java (Spring), etc.
- **Database** — the storage layer, where data is persisted. Relational (MySQL, PostgreSQL) or NoSQL (MongoDB, Redis).

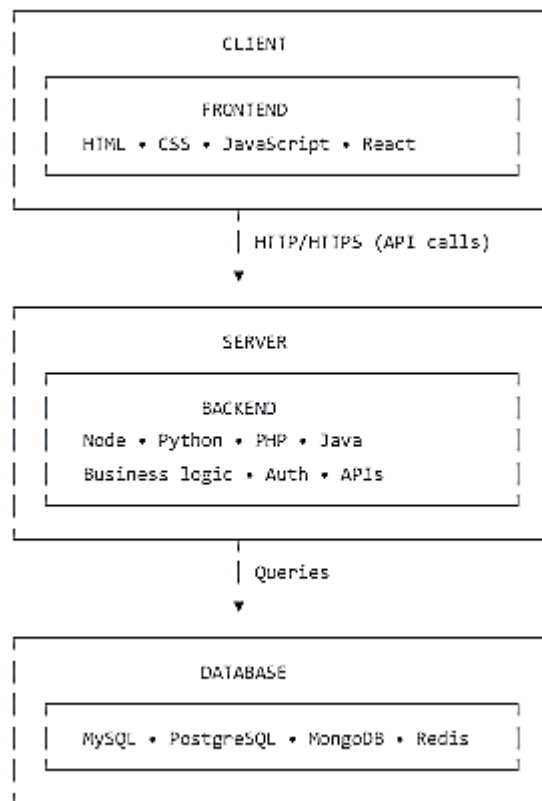


Figure 5: Three-Tier Architecture

2.3.6 APIs Introduction

API (Application Programming Interface) is a set of rules that allows two software programs to communicate with each other. In web development, APIs typically allow the frontend to talk to the backend, or allow one service to talk to another (e.g., a payment gateway, weather service, or maps).

Table 3: Types of APIs and Their Characteristics

Type	Description	Example
REST	Uses HTTP methods, stateless, most common	Twitter API, Stripe
GraphQL	Query language, client asks for exactly what it	GitHub

Type	Description	Example
	needs	API v4
SOAP	Older XML-based protocol, enterprise use	PayPal (legacy)
WebSockets	Real-time, bidirectional communication	Chat apps, live scores

How it works:

1. Frontend sends an HTTP request to an API endpoint (e.g., GET/api/users/1).
2. Backend processes it, queries the database.
3. Backend returns a response (usually JSON).
4. Frontend uses that data to update the UI.

Table 4: REST API Methods and Endpoints

Method	Endpoint	Purpose
GET	/api/users	List all users
GET	/api/users/1	Get user with ID 1
POST	/api/users	Create a new user
PUT	/api/users/1	Replace user 1
PATCH	/api/users/1	Update user 1 partially
DELETE	/api/users/1	Delete user 1

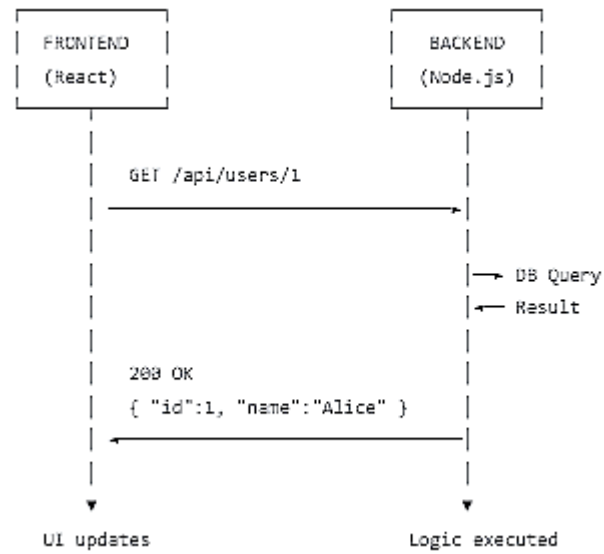


Figure 6: API as a Bridge Between Frontend and Backend

CHAPTER 3: WEB DEVELOPMENT TECHNOLOGIES

3.1 Frontend Technologies

HTML provides the basic structure and content of webpages using elements such as headings, paragraphs, links, images, forms, and tables.

CSS controls the appearance and layout of webpages, including colors, fonts, spacing, animations, and responsive design. Modern CSS uses Flexbox and Grid for page layouts.

JavaScript adds interactivity and dynamic behavior to websites through features such as form validation, event handling, DOM manipulation, API requests, and dynamic content.

For complex applications, developers can use frameworks and libraries such as React, Angular, Vue, Svelte, and Next.js. The choice of technology should be based on project requirements, team skills, maintainability, and technical needs.

3.2 Backend Technologies

Common backend technologies include:

JavaScript/Node.js — allows JavaScript to execute outside the browser and is widely used for server-side applications and APIs.

Python — can be used with frameworks such as Django, Flask, and FastAPI.

Java — widely used for enterprise applications and backend systems.

PHP — a server-side language used for web development and powers many websites and web systems.

C# — with the .NET ecosystem can be used to develop web applications and APIs.

3.3 Databases and Version Control

The choice of database depends on application requirements.

A relational database represents structured information using tables. Examples include PostgreSQL, MySQL, Oracle Database, and Microsoft SQL Server. These databases commonly use SQL.

A NoSQL database may represent information using documents or other non-relational models. Examples include MongoDB, Redis, and Cassandra.

Git is a distributed version-control system used to track changes in source code. Important Git concepts include repository, commit, branch, merge, clone, pull, push, and remote repository. Version control is important because it allows developers to maintain project history, collaborate with other developers, and recover previous versions when required.

3.4 Development Tools and Deployment

Developers use code editors and integrated development environments to write and manage source code. Examples include Visual Studio Code, Visual Studio, IntelliJ IDEA, WebStorm, and PyCharm. Browser developer tools are also important for inspecting HTML and CSS, debugging JavaScript, analyzing network requests, and investigating performance.

Package managers help developers install and manage software dependencies. Examples include npm, pnpm, Yarn, pip, Maven, and Composer.

After development and testing, a web application can be deployed so that users can access it. Deployment may involve web hosting, cloud platforms, domain configuration, DNS, SSL/TLS certificates, environment variables, databases, CI/CD pipelines, and containers.

CHAPTER 4: CAREER ROADMAP FOR WEB DEVELOPMENT IN 2026

4.1 Roadmap Overview

A career roadmap provides a structured sequence for developing technical and professional skills. The proposed roadmap is not a rigid requirement. Learners may change the order or specialize depending on their goals, educational background, available resources, and career interests.

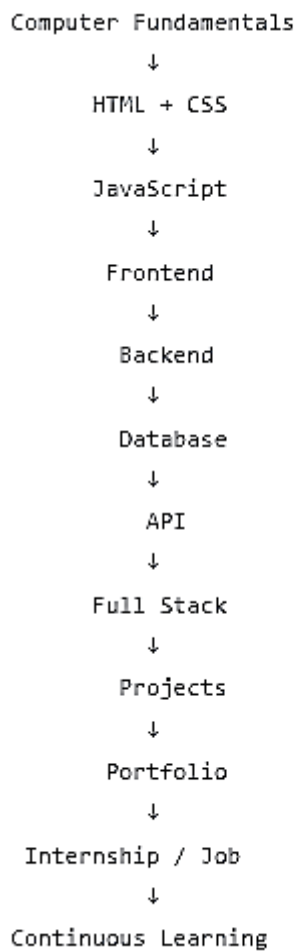


Figure 7: Web Development Career Roadmap

4.2 Stages 1–5: Foundations to Backend

Stage 1: Computer and Web Fundamentals: Beginners should first understand computer fundamentals, operating systems, files and folders, internet fundamentals, browsers, URLs, HTTP, basic networking concepts, and command-line basics. This foundation makes later technical concepts easier to understand.

Stage 2: HTML and CSS: This stage focuses on HTML document structure, semantic HTML, forms, tables, links, images, and accessibility basics, along with CSS selectors, box model, Flexbox, Grid, responsive design, media queries, transitions, and basic animations. At the end of this stage, learners should be able to create responsive static websites.

Stage 3: JavaScript: Important concepts include variables, data types, operators, conditions, loops, functions, arrays, objects, DOM, events, modules, asynchronous programming, promises, fetch/API requests, and error handling.

Stage 4: Frontend Development: After learning JavaScript fundamentals, learners can study a frontend library or framework. Possible areas include components, props, state, routing, forms, API integration, authentication, state management, and reusable UI components. The learner should focus on understanding the underlying JavaScript and web concepts rather than relying completely on a framework.

Stage 5: Backend Development: Backend learning may include server fundamentals, routing, controllers, middleware, authentication, authorization, validation, error handling, file handling, and API development. A learner can choose one backend ecosystem and develop practical projects with it.

4.3 Stages 6–10: Database to Continuous Learning

Stage 6: Database Development: Database skills should include tables, relationships, primary keys, foreign keys, constraints, SQL, joins, aggregation, transactions, indexing, and basic database security. Learners can begin with one relational database and later explore other database models.

Stage 7: Full-Stack Development: Full-stack development combines frontend, backend, and database technologies. A basic full-stack system can be represented as:

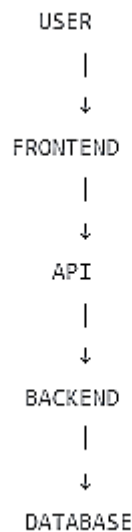


Figure 8: Full-Stack System

The learner should understand how data moves across these layers.

Stage 8: Projects and Portfolio: Learners should apply their skills through practical projects, starting with simple applications and progressing to advanced full-stack or business systems. Each project should demonstrate specific technical skills.

Stage 9: Internship and Job Preparation: After building foundational skills and projects, learners can prepare for professional opportunities. Preparation may include resume preparation, portfolio development, GitHub profile, LinkedIn profile, technical interview preparation, problem-solving practice, communication skills, project explanation, mock interviews, and internship applications.

Stage 10: Continuous Learning: Web development changes continuously. Therefore, professional development should continue after obtaining a job. Developers may need to learn new frameworks, cloud services, DevOps practices, security, testing, performance optimization, AI-assisted development tools, accessibility, and system design. The fundamental concepts should remain the foundation for learning new technologies.

CHAPTER 5: SKILLS AND CAREER OPPORTUNITIES

5.1 Technical and Professional Skills

Web developers need technical skills such as HTML, CSS, JavaScript, frontend and backend development, databases, APIs, Git, testing, debugging, deployment, and security. Problem-solving, technical communication, documentation, teamwork, task management, and time management are also essential for professional software development.

5.2 Career Opportunities

Web development provides various career paths, including Frontend Developer, Backend Developer, Full-Stack Developer, UI Developer, Software Developer, Freelancer, Remote Developer, and Entrepreneur. These roles involve developing interfaces, server-side systems, APIs, databases, software applications, and digital products.

5.3 Career Preparation

Students can prepare for careers by building a strong resume, portfolio, GitHub and LinkedIn profiles, practicing technical and problem-solving skills, preparing for interviews, improving communication, participating in internships, and developing real-world projects that demonstrate their abilities.

CHAPTER 6: METHODOLOGY AND IMPLEMENTATION

6.1 Research Method and Information Gathering

The project uses a descriptive and applied research method. Information about web development was collected from credible sources, organized into a logical structure, simplified for a beginner audience, and applied to develop both a technical report and an educational webinar.

Information was gathered from official technical documentation (MDN Web Docs, W3C, Git Documentation), industry surveys (Stack Overflow Developer Survey), security standards (OWASP Top 10), and academic and standards bodies (IEEE, ISO/IEC). Official documentation was preferred over blogs, videos, and informal tutorials because it is authoritative, regularly maintained, and technically accurate. All key information was cross-checked against at least two independent sources before inclusion in the report.

6.2 Webinar Implementation

The webinar was developed from the technical report. The presentation followed a logical sequence: title slide, learning objectives, web fundamentals, frontend, backend, database, APIs, development workflow, career roadmap, required skills, career opportunities, Q&A, and closing slide.

The webinar was conducted through an online communication platform. The presenter introduced the topic, explained the technical concepts, demonstrated selected examples, discussed the career roadmap, and provided an opportunity for participants to ask questions.

Table 5: Tentative Webinar Schedule

Activity	Duration
Introduction and Welcome	10 minutes
Fundamentals of Web Development	20 minutes

Activity	Duration
Frontend, Backend, and Database	25 minutes
Career Roadmap for 2026	20 minutes
Question and Answer Session	10 minutes
Feedback Collection and Closing	5 minutes
Total	90 minutes

6.4 Resources and Risk Management

Hardware Resources: Computer or laptop, microphone, webcam, headphones, stable internet connection.

Software Resources: Web browser, Visual Studio Code or another code editor, Git, presentation software, screen-recording software, image/diagram creation tools.

Online Resources: Google Meet or Zoom, online documentation, Git repository platform, online feedback form, cloud storage.

Table 6: Webinar Risk Management Plan

Risk	Probability	Impact	Prevention
Internet failure during webinar	Medium	High	Backup connection; record session
Platform technical issues	Low	High	Test platform beforehand; alternative platform ready
Low participant attendance	Medium	Medium	Promote early; send reminders
Audio/video quality	Medium	Medium	Test equipment; use quality

Risk	Probability	Impact	Prevention
issues			microphone
Time overrun	Medium	Medium	Rehearse; strict timekeeping
Participant disengagement	Low	Medium	Interactive Q&A; polls

CHAPTER 7: CONCLUSION AND RECOMMENDATIONS

7.1 Conclusion

Web development is a broad and continuously evolving field that includes frontend, backend, databases, APIs, testing, deployment, security, and software engineering. This technical writing project provides beginners with a structured introduction to these concepts and presents a practical career roadmap for 2026. The roadmap guides learners from basic computer and web fundamentals through HTML, CSS, JavaScript, frontend and backend development, databases, APIs, projects, portfolio development, and career preparation.

The project also highlights the importance of technical communication, documentation, teamwork, and presentation skills in software development. The webinar provides an effective way to share this knowledge through presentation, discussion, demonstrations, and Q&A sessions. Overall, the project helps learners understand web development in a structured and practical way while fulfilling the objectives of the Technical Writing course.

7.2 Recommendations

Beginners should first develop a strong foundation in HTML, CSS, and JavaScript before moving to advanced frameworks and technologies. They should focus on understanding programming concepts and apply their knowledge through practical projects. Learning Git, databases, APIs, testing, and security should be part of the development journey.

Students should also create a portfolio that demonstrates their projects and skills, while improving technical communication and documentation abilities. Using official documentation and reliable learning resources is recommended for accurate and up-to-date knowledge. Finally, learners should continue updating their skills because web technologies, tools, and industry practices change over time.

Chapter 8: REFERENCES

1. MDN Web Docs. *Web Technology for Developers*. Mozilla Developer Network. [Online]. Available: <https://developer.mozilla.org>
2. MDN Web Docs. *How the Web Works*. Mozilla Developer Network. [Online]. Available: <https://developer.mozilla.org>
3. MDN Web Docs. *Introduction to Web APIs*. Mozilla Developer Network. [Online]. Available: <https://developer.mozilla.org>
4. Git Documentation. *Git Documentation and Getting Started Guide*. Git Project. [Online]. Available: <https://git-scm.com/doc>
5. Stack Overflow. *2025 Developer Survey: Technology*. Stack Overflow. [Online]. Available: <https://survey.stackoverflow.co>